AI Agents

A Comprehensive Technical Guide

Introduction • Components • ReAct • Conversational Agents • Persistent Memory

Enterprise AI Bootcamp Series

Topics Covered	5 Major Modules
Audience	Fresher / Entry-Level AI Engineers
Frameworks	LangChain, LangGraph, Mem0
Patterns	ReAct, Deep Agents, Memory Lifecycle

Table of Contents

Module 1: Introduction to Agents

1.1 What is an AI Agent?

An AI Agent is an autonomous software entity that perceives its environment, reasons about the perceived information, decides on a course of action, and executes that action — often in a loop until a goal is achieved. Unlike a simple chatbot that produces a single response, an agent can break a complex task into steps, call external tools, store intermediate results, and adjust its plan based on new observations.

The key differentiator is **autonomy**: an agent decides *what* to do next without a human providing step-by-step instructions. This makes agents suitable for open-ended, multi-step tasks such as research, data analysis, code generation, customer support workflows, and business automation.

Core Properties of an AI Agent

- Perception — receives inputs (text, tool results, sensor data)
- Reasoning — uses an LLM or rule engine to interpret and plan
- Action — invokes tools, APIs, databases, or other agents
- Memory — stores context across turns and tasks
- Goal-directedness — optimises toward an objective over multiple steps

1.2 LLM + Tools + Memory + Planning

Modern AI agents are built on four interconnected pillars. The Large Language Model (LLM) acts as the cognitive core that understands natural language, generates plans, interprets tool outputs, and produces final answers.

Pillar	Role in the Agent
LLM (Brain)	Generates text, reasons about goals, selects tools, summarises results.
Tools	Functions the LLM can call: search engines, calculators, SQL queries, REST APIs, file I/O.
Memory	Short-term (current conversation), long-term (vector stores, databases), episodic (past task logs).
Planning	Decomposing goals into sub-tasks, ordering steps, handling failures, and re-planning dynamically.

Together these four pillars allow an agent to behave like a knowledgeable assistant that can look things up, remember previous interactions, and adapt its strategy when something goes wrong.

1.3 Agent vs Workflow

A common source of confusion is the distinction between an **agent** and a **workflow**. Both orchestrate multiple steps, but they differ fundamentally in who — or what — controls the sequencing.

Dimension	Workflow	Agent
Control	Pre-defined DAG / script	LLM decides next step dynamically
Flexibility	Rigid; steps are fixed	Adaptive; re-plans on new information
Tooling	Usually deterministic calls	Selects tools autonomously
Failure	Halts or follows error branch	Can retry, change strategy, ask user
Example	ETL pipeline, CI/CD build	Research assistant, coding copilot
Latency	Low and predictable	Higher; LLM calls add overhead
Best for	Structured, repetitive tasks	Open-ended, ambiguous tasks

Key Insight

- Use a Workflow when you know every step in advance and need reliability.
- Use an Agent when the task is open-ended, multi-step, or requires adapting to unpredictable information.
- Hybrid systems (agentic workflows) are increasingly common: a workflow orchestrates agents that handle specific sub-tasks autonomously.

1.4 Introduction to Modern Agent Frameworks

Several open-source and commercial frameworks have emerged to simplify building production-grade AI agents. Each makes different trade-offs between flexibility, abstraction level, and ecosystem.

Framework	Key Strengths	Primary Use Case
LangChain	Huge tool ecosystem, chains, agents, memory abstractions	Rapid prototyping, RAG, tool-using agents
LangGraph	Stateful graph-based orchestration, cycles, checkpointing	Complex multi-step agents, human-in-the-loop
AutoGen	Multi-agent conversations, code execution sandboxes	Collaborative agent networks
CrewAI	Role-based multi-agent crews, task delegation	Business process automation with agent teams
Semantic Kernel	Microsoft ecosystem, plugins, planners	.NET / Azure enterprise agents
Haystack	NLP pipelines, RAG, document agents	Document Q&A, search agents

1.5 Deep Agents Concept in LangChain

LangChain introduced the concept of **Deep Agents** — agents that go beyond simple Thought→Action→Observation loops to support complex, multi-turn, stateful execution graphs. Implemented primarily through **LangGraph**, Deep Agents model agent behaviour as a directed (possibly

cyclic) graph of nodes and edges.

- **Nodes** represent discrete processing steps: LLM calls, tool invocations, human checkpoints, sub-agents.
- **Edges** are conditional: the agent decides which node to visit next based on state.
- **State** is a typed dictionary that flows through the graph and accumulates results.
- **Persistence** is built-in via checkpointers (SQLite, Redis, Postgres) allowing agents to pause and resume.
- **Interrupts** allow human-in-the-loop validation at any graph node.
- **Streaming** support lets intermediate steps surface to the user in real time.

This architecture addresses a core limitation of simple ReAct agents: the inability to maintain rich state across long, branching task sequences. LangGraph Deep Agents are used for production deployments that require reliability, auditability, and complex control flow.

Module 2: Agent Components

2.1 Tools (API / DB)

Tools are the hands of an AI agent — the mechanisms through which the agent acts on the world beyond pure text generation. A tool is a callable function that the LLM can invoke by producing a structured JSON payload.

Categories of Tools

Category	Examples	What the Agent Gets
Web / Search	Tavily, Serper, DuckDuckGo	Live web pages, news, research data
Database	SQL executor, MongoDB, Redis	Structured query results, counts, records
REST API	Weather API, CRM, payment gateway	JSON responses from third-party services
Code Execution	Python REPL, Jupyter kernel, bash shell	Stdout, stderr, computed values
File System	Read/write files, parse PDFs, CSV loader	Document content, transformed files
Vector Store	FAISS, Pinecone, Weaviate similarity search	Semantically relevant document chunks
Communication	Email sender, Slack, calendar APIs	Confirmation, scheduling results

Defining a Tool in LangChain

```
from langchain.tools import tool @tool def search_database(query: str) -> str:
"""Search the product database for items matching the query.""" results =
db.execute(f"SELECT * FROM products WHERE name LIKE '%{query}%'") return
str(results.fetchall())
```

- Tools must have a clear docstring — the LLM reads this to know when to call the tool.
- Input/output should be simple types (str, int, dict); complex objects cause serialisation errors.
- Always add error handling inside tools: the agent relies on tool output to make decisions.
- Tools can themselves call other agents (tool-calling agents as sub-agents).

2.2 Memory

Memory allows an agent to retain and recall information across turns, tasks, and even sessions. Without memory, every agent interaction starts from a blank slate, making it impossible to build personalised or multi-session workflows.

Memory Type	Description & Storage
Conversational (Buffer)	Keeps all messages in the LLM context window. Simple but expensive for long conversations.

Summary Memory	Periodically summarises older turns to compress the context. Loses some detail.
Entity Memory	Extracts and tracks named entities (people, places, concepts) mentioned by the user.
Vector Store Memory	Embeds messages and retrieves semantically similar ones via cosine similarity at each turn.
External / Persistent	Writes to Redis, PostgreSQL, or cloud storage. Survives process restarts. Used in Deep Agents.

2.3 Planning

Planning is the agent's ability to decompose a high-level goal into an ordered sequence of achievable sub-tasks and to revise that plan when obstacles arise.

- **Task Decomposition** — Break 'write a market report' into: research, outline, write sections, edit, format.
- **Chain-of-Thought (CoT)** — The LLM reasons step-by-step before committing to a tool call.
- **Tree-of-Thought (ToT)** — Explores multiple reasoning branches simultaneously, picks the best path.
- **Plan-and-Execute** — A planner LLM creates a full plan upfront; an executor LLM executes each step.
- **ReWOO** — Separates reasoning from tool execution to reduce LLM calls and latency.
- **Dynamic Re-planning** — Agent revisits the plan after each observation and adjusts remaining steps.

2.4 Reflection Loops

Reflection is a meta-cognitive capability: the agent reviews its own outputs, identifies errors or gaps, and attempts to improve before presenting the final answer. This is inspired by the human practice of drafting, reviewing, and revising.

- **Self-critique** — Agent prompts itself: 'Does my answer address all parts of the question?'
- **Critic agent** — A separate LLM instance (or a different system prompt) evaluates the primary agent's output.
- **Reflexion pattern** — Agent stores episodic memory of past failures and consults it before acting.
- **Constitutional AI** — Agent checks its output against a set of principles before responding.
- **Loop termination** — Reflection loops must have a maximum iteration count to avoid infinite loops.

Reflection Loop Pseudocode

- 1. Agent produces initial answer
- 2. Critic LLM (or same LLM with critique prompt) evaluates the answer
- 3. If quality < threshold AND iterations < max_iter: revise and go to step 2
- 4. Else: return final answer

2.5 Structured Execution in Deep Agents (LangGraph)

LangGraph's graph-based model gives Deep Agents a structured, inspectable execution model that is fundamentally different from the linear prompt-response loops of simpler agents.

- **TypedDict State** — All agent state is captured in a strongly typed dictionary shared across nodes.
- **Conditional Edges** — After each node, a router function determines the next node based on state.
- **Checkpointing** — State snapshots are saved after every node; the agent can resume from any checkpoint.
- **Subgraphs** — Complex agents are composed of nested graphs, each handling a specific concern.
- **Parallel Execution** — Nodes with no dependency can execute concurrently (map-reduce patterns).
- **Human-in-the-loop** — An interrupt() call pauses execution for human review before proceeding.

Module 3: The ReAct Pattern

3.1 Thought → Action → Observation Loop

ReAct (Reasoning + Acting) is a foundational agent paradigm introduced by Yao et al. (2022). It interleaves language model reasoning traces (*thoughts*) with external actions, creating a transparent, interpretable loop that combines the benefits of chain-of-thought reasoning with grounded tool use.

The loop operates as follows:

- **Thought** — The LLM articulates its current understanding, goal, and next intended action in natural language.
- **Action** — The LLM emits a structured tool-call instruction (e.g., Search[quantum computing]).
- **Observation** — The tool executes and returns its result; this is appended to the context.
- **Repeat** — The LLM reads the observation and produces the next Thought, continuing until it can emit a Final Answer.

```
Example ReAct Trace Question: What is the population of the capital of France? Thought: I need to find the capital of France, then look up its population. Action: Search[capital of France] Observation: The capital of France is Paris. Thought: Now I need the current population of Paris. Action: Search[population of Paris 2024] Observation: Paris has a population of approximately 2.1 million (city proper). Thought: I have the answer. Final Answer: The capital of France is Paris with a population of ~2.1 million.
```

The key strength of ReAct is **transparency**: each reasoning step is visible, making it easy to debug failures and understand agent behaviour. It also naturally handles multi-hop questions that require chaining multiple tool calls.

3.2 Limitations of ReAct

Despite its elegance, ReAct has several practical limitations that motivated the development of more sophisticated agent architectures:

Limitation	Detail
No persistent state	Each run starts fresh; there is no built-in mechanism to carry state across sessions or tasks.
Context window pressure	Every thought, action, and observation is appended to the context. Long tasks overflow the window.
No structured control flow	Loops, conditionals, and parallelism must be implicitly reasoned by the LLM — fragile and inconsistent.

Hallucination in reasoning	The LLM may 'think' it took an action it did not, or misinterpret an observation.
No recovery planning	When a tool fails, the agent often gets confused rather than systematically retrying or changing strategy.
Single-threaded	Cannot execute independent sub-tasks in parallel; every step is sequential.
No memory hierarchy	ReAct has no notion of short vs long-term memory; everything is flat context.

3.3 ReAct vs Structured / Deep Agents

The following comparison positions ReAct against more advanced architectures such as LangGraph Deep Agents and Plan-and-Execute systems:

Feature	ReAct	Deep Agent (LangGraph)
State management	Implicit in context string	Explicit TypedDict, persisted
Control flow	LLM decides implicitly	Coded graph nodes + edges
Memory	None (context window only)	Multi-tier: buffer, vector, external
Parallelism	None	Parallel node execution
Checkpointing	None	Built-in (SQLite/Redis)
Human-in-the-loop	Manual integration	Native interrupt() support
Debuggability	Read trace text	Visualise graph; inspect state
Scalability	Limited by context window	Scales with external storage
Best for	Simple Q&A, demos, prototypes	Production, multi-step, long-horizon

Module 4: Conversational Agents

4.1 Stateful Conversations

A stateful conversational agent maintains an evolving understanding of the dialogue history, user intent, and task progress across multiple turns. This is fundamentally different from stateless chatbots that treat each message in isolation.

- **Turn tracking** — Agent knows which turn it is on, enabling references like 'as I mentioned earlier'.
- **Intent persistence** — User's goal established in turn 1 remains active unless explicitly changed.
- **Slot filling** — Agent progressively collects required information (name, date, preference) across turns.
- **Clarification handling** — When the user is ambiguous, agent asks targeted follow-up questions.
- **Task resumption** — With persistent storage, agent can continue a multi-day task across sessions.

4.2 Context Retention

Context retention is the mechanism by which an agent preserves relevant information from earlier in a conversation so it can be used later. The challenge is that LLM context windows are finite, so agents must decide what to keep, what to compress, and what to offload.

Context Retention Strategies

- Full buffer — Keep all messages verbatim (best for short conversations, ≤ 20 turns)
- Sliding window — Retain only the last N turns; oldest messages are dropped
- Summary buffer — LLM periodically summarises old turns; summary replaces raw messages
- Token-based trimming — Drop messages when total tokens exceed a threshold
- Selective retrieval — Embed all messages; retrieve only semantically relevant ones at each turn

4.3 Memory Strategies in Agents

Beyond raw context retention, conversational agents use structured memory strategies to organise and retrieve information intelligently:

Strategy	Mechanism & Trade-offs
ConversationBufferMemory	Appends every message. Simple; blows up context for long chats.
ConversationSummaryMemory	Generates a running summary. Loses details; suitable for very long sessions.

ConversationSummaryBufferMemory	Keeps recent turns verbatim + summary of older turns. Best of both worlds.
ConversationEntityMemory	Extracts entities (people, products, dates) into a structured KB. Enables personalised responses.
VectorStoreRetrieverMemory	Embeds all turns; retrieves top-k most relevant at query time. Scales to very long histories.
Redis / PostgreSQL	Serialises conversation state to external DB. Enables cross-session and cross-device continuity.

4.4 Memory Strategies in Deep Agents (LangGraph)

LangGraph extends memory management with first-class primitives that integrate directly into the agent's execution graph:

- **State as memory** — The graph's TypedDict state *is* the agent's working memory. Every node can read and write to it.
- **Checkpointers** — MemorySaver, SqliteSaver, PostgresSaver persist state after every node, enabling agents to resume exactly where they left off.
- **Thread-scoped memory** — Each conversation has a unique thread_id; memory is isolated per thread.
- **Cross-thread memory (Store)** — LangGraph's Store API enables sharing facts across threads (e.g., user preferences learned in thread A are available in thread B).
- **Memory as nodes** — Agents can have dedicated 'memory write' and 'memory read' nodes that explicitly manage long-term knowledge.
- **Namespace management** — Store entries are organised by namespace tuples (e.g., (user_id, 'preferences')) for efficient retrieval.

Module 5: Persistent Memory Systems — Mem0 Concepts

5.1 Beyond Chat Memory

Traditional agent memory is conversation-scoped: it exists only within a single session and is discarded when the session ends. Mem0 (pronounced 'mem-zero') is an open-source intelligent memory layer that gives AI agents long-term, cross-session, user-aware memory — similar to how a human professional remembers past clients.

Mem0 sits as a layer between the user and the LLM. As conversations proceed, Mem0 automatically extracts important facts, preferences, and events, stores them in a hybrid vector + key-value store, and injects relevant memories into future prompts — without the developer having to manually manage any of this.

What Mem0 Enables That Chat Memory Cannot

- User preferences persist across sessions (e.g., 'prefers concise answers')
- Facts learned in session 1 are available in session 100
- Multi-user memory: each user has their own memory namespace
- Memory evolution: facts can be updated when new information contradicts old
- Memory importance scoring: critical facts are retained; trivial chatter is pruned
- Semantic search over memory: 'what do I know about Alice's dietary needs?'

5.2 User Preference Learning

One of Mem0's most powerful applications is **automatic user preference learning**. As the user interacts with the agent, Mem0 extracts implicit and explicit preferences from the conversation and stores them as structured memory entries.

- **Explicit preferences** — User directly states: 'I prefer Python over JavaScript.'
- **Implicit preferences** — User consistently asks for bullet-point answers → agent infers a formatting preference.
- **Domain preferences** — User always discusses ML in the context of healthcare → domain tagged in memory.
- **Preference conflicts** — If user states a new preference contradicting an old one, Mem0 updates the entry.
- **Preference injection** — On subsequent turns, relevant preferences are prepended to the system prompt.

```
# Mem0 preference extraction example from mem0 import Memory m = Memory() # User message triggers automatic extraction m.add('I always prefer concise bullet-point answers', user_id='user_42') # Later, retrieve relevant memories before calling LLM relevant = m.search('how should I format my response?', user_id='user_42') # Returns:
```

```
[{'memory': 'Prefers concise bullet-point answers', 'score': 0.94}]
```

5.3 Memory Extraction

Memory extraction is the process of identifying **what** is worth remembering from a conversation. Mem0 uses an LLM-based extraction pipeline that analyses messages and produces structured memory entries.

- **Fact extraction** — Identify declarative statements: 'I live in Mumbai', 'I am a data engineer'.
- **Event extraction** — Capture time-bound events: 'User completed the ML course on 15 March'.
- **Preference extraction** — Detect likes, dislikes, communication styles.
- **Relationship extraction** — Note entities and their relationships: 'Alice is the user's manager'.
- **Contradiction detection** — If new message contradicts existing memory, flag for update rather than duplicate creation.
- **Granularity control** — Developer can configure what categories of information to extract.

Extraction Prompt Pattern

```
System: You are a memory extraction engine. Analyse the following conversation and
extract ONLY factual, preference, or event-based information worth remembering
long-term. Output a JSON list of memory entries: [{"memory": str, "category": str,
"entities": [str]}] Ignore small talk, greetings, and transient information.
```

5.4 Importance Scoring

Not all information deserves to be remembered. Mem0 assigns an **importance score** (0.0 – 1.0) to each extracted memory entry. This score determines retention priority during memory pruning.

Factor	Impact on Importance Score
Recency	More recently mentioned facts receive a higher base score; score decays over time.
Frequency	Facts mentioned multiple times across sessions score higher — user clearly cares.
Explicitness	Directly stated facts ('I am vegetarian') score higher than inferred ones.
Domain relevance	Facts directly relevant to the agent's purpose score higher.
User confirmation	If agent reflects a memory back and user confirms it, score is boosted.
Semantic uniqueness	Facts with no similar entries in memory score higher (novel information).

5.5 Long-Term Personalisation

Long-term personalisation is the end goal of a persistent memory system: an agent that becomes demonstrably more useful the more a user interacts with it. Mem0 achieves this through accumulation, consolidation, and contextual injection of memories.

- **Progressive personalisation** — After 5 sessions, agent knows your name and role; after 50, it knows your working style, preferences, and recurring challenges.
- **Memory-augmented prompts** — Before every LLM call, Mem0 searches for relevant memories and prepends them to the system prompt automatically.
- **Persona-aware responses** — Agent adapts tone, depth, and format based on remembered user profile.
- **Proactive assistance** — Agent can reference past context unprompted: 'Last time you asked about this, you ran into an issue with X.'
- **Cross-domain memory** — Memories from one task context (data engineering) inform another (general coding advice) if relevant.

5.6 Memory Lifecycle: Store → Update → Prune

Mem0 manages a full memory lifecycle. Each entry passes through three stages: creation (Store), revision (Update), and removal (Prune). This lifecycle ensures the memory store remains accurate, relevant, and computationally tractable.

Stage 1: Store

When a new piece of information is extracted from a conversation, Mem0 attempts to store it. Before writing, it checks for semantic duplicates to avoid redundancy.

- New conversation turn is sent to the extraction pipeline.
- LLM identifies memory-worthy facts, preferences, and events.
- Each candidate is embedded (using an embedding model like text-embedding-3-small).
- Cosine similarity search against existing memories is performed.
- If similarity < threshold (e.g., 0.85): write as a new entry with metadata (user_id, timestamp, category, importance_score).
- If similarity >= threshold: route to the Update stage instead.

```
# Mem0 store operation memory.add( messages=conversation_history, user_id='user_42', metadata={'session_id': 'sess_007', 'source': 'chat'} ) # Mem0 internally: # 1. Calls LLM to extract facts # 2. Embeds each fact # 3. Checks for duplicates # 4. Writes to vector store + key-value store
```

Stage 2: Update

When incoming information is semantically similar to an existing memory (suggesting it is related) or directly contradicts it (suggesting the old information is stale), Mem0 updates rather than duplicates.

- **Refinement update** — New info adds detail to existing entry: 'User is a data engineer at TechCorp' updates 'User is a data engineer'.
- **Contradiction update** — LLM detects conflict and overwrites: 'User now prefers verbose answers' replaces 'User prefers concise answers'.
- **Temporal update** — Date-stamped facts are updated when a more recent version is observed: 'User is currently learning PyTorch' replaces the same from 3 months ago.
- **Confidence adjustment** — If a memory was previously inferred and user now explicitly confirms it, confidence score is upgraded.
- **Audit log** — Original value, update timestamp, and reason are preserved in metadata for explainability.

```
# Mem0 update example (automatic) # Old memory: {'id': 'mem_123', 'text': 'User prefers Python', 'score': 0.7} # New conversation: 'Actually I mainly use Python for ML but Go for backends' # After update: # {'id': 'mem_123', # 'text': 'User prefers Python for ML tasks, Go for backend development', # 'score': 0.85, # 'updated_at': '2025-04-01T10:22:00Z', # 'previous': 'User prefers Python'}
```

Stage 3: Prune

Memory stores grow over time and must be pruned to maintain relevance, control costs, and respect privacy. Mem0's pruning mechanism uses multiple signals to decide which memories to evict.

- **Score-based pruning** — Entries with importance score below a configurable threshold are candidates for deletion.
- **Time-decay pruning** — Memories that have not been accessed (retrieved) for N days have their scores decayed; eventually they fall below the threshold.
- **Storage-budget pruning** — When the memory store exceeds a size limit, lowest-scoring entries are evicted first.
- **User-triggered pruning** — User explicitly asks to forget something; agent calls memory.delete() with the relevant entry IDs.
- **Category-based pruning** — Entire categories of memory (e.g., 'session_notes') can be pruned on schedule while 'preferences' are retained.
- **Consolidation before pruning** — Before deleting related low-score entries, Mem0 attempts to consolidate them into a single higher-level summary entry.

Pruning Strategy	When to Use
Recency-based (LRU)	Evict memories not accessed in the last 30/60/90 days. Good for session-specific facts.
Score threshold	Delete entries with importance < 0.3. Aggressive but keeps store lean.
Duplicate merging	Cluster similar entries; keep the highest-scoring representative, delete others.
GDPR / explicit delete	User requests data deletion. All entries for user_id are hard-deleted.
Scheduled batch pruning	Cron job runs nightly to recalculate scores and prune below threshold.

Memory Lifecycle Summary

- STORE → Extract facts from conversation → embed → deduplicate → write with metadata
- UPDATE → Detect contradictions or refinements → merge → log previous value → update score
- PRUNE → Decay scores over time → apply threshold → consolidate → evict lowest-priority entries
- INJECT → At query time, semantic search → top-k retrieval → prepend to system prompt

Summary & Key Takeaways

Concept Map

The five modules of this guide build on each other progressively:

- Module 1 established the foundation: what agents are, their four pillars (LLM, Tools, Memory, Planning), and how they differ from static workflows.
- Module 2 drilled into each pillar as a buildable component, covering tool design patterns, memory types, planning strategies, and reflection loops.
- Module 3 examined the ReAct pattern — the simplest viable agent loop — and its limitations that motivated richer architectures.
- Module 4 explored conversational agents: how state is maintained across turns, and how LangGraph's Deep Agent model manages memory natively.
- Module 5 introduced Mem0's persistent memory system: the full lifecycle of memory from extraction and importance scoring through updates and pruning, enabling true long-term personalisation.

Quick Reference: When to Use What

Scenario	Recommended Approach	Key Technology
Simple Q&A chatbot	Stateless LLM + prompt	OpenAI API, Anthropic API
Single-session assistant	ConversationBufferMemory	LangChain ConversationChain
Long single-session task	SummaryBufferMemory + ReAct agent	LangChain AgentExecutor
Multi-step complex task	LangGraph Deep Agent	LangGraph StateGraph
Multi-session personalisation	Persistent memory (Mem0)	Mem0, Redis, Postgres
Multi-agent collaboration	Agent crews / supervisor pattern	CrewAI, LangGraph multi-agent
Human-in-the-loop workflows	LangGraph with interrupt()	LangGraph + checkpointer
Production agent with auditability	LangGraph + external checkpointer	LangGraph + PostgresSaver

Recommended Learning Path

- Step 1 — Understand LLM basics (prompt engineering, tool calling, function calling)
- Step 2 — Build a simple ReAct agent with LangChain AgentExecutor + 2-3 tools
- Step 3 — Add ConversationSummaryBufferMemory to your agent
- Step 4 — Rebuild the same agent in LangGraph as a StateGraph to understand Deep Agents
- Step 5 — Add LangGraph checkpointing (MemorySaver) for persistence
- Step 6 — Integrate Mem0 to add cross-session user preference learning
- Step 7 — Build a multi-agent graph with a supervisor routing to specialist sub-agents

- Step 8 — Add human-in-the-loop interrupts and deploy with LangServe or FastAPI

This guide is part of the Enterprise AI Bootcamp Series. All code examples target Python 3.10+, LangChain 0.3.x, LangGraph 0.2.x, and Mem0 0.1.x. Always consult official documentation for the latest APIs.